Document number:	P1194
Date:	2018-10-08
Title:	**The Compromise Executors Proposal: A lazy simplification of P0443**
Audience:	SG1 & LEWG
Reply-to:	Lee Howes <lwh@fb.com (mailto:lwh@fb.com)> Eric Niebler <eniebler@fb.com (mailto:eniebler@fb.com)> Kirk Shoop <kirkshoop@fb.com (mailto:kirkshoop@fb.com)> Bryce Lelbach <brycelelback@gmail.com (mailto:brycelelback@gmail.com)> David S. Hollman <dshollm@sandia.gov (mailto:dshollm@sandia.gov)>

1 Summary

This paper seeks to add support for lazy task creation and deferred execution to P0443 (https://wg21.link/P0443), while also simplifying the fundamental concepts involved in asynchronous execution. It seeks to do this as a minimal set of diffs to P0443. It achieves this by replacing P0443's six `Executor::*execute` member functions with two lazy task constructors that each return a (potentially) lazy Future-like type known as a "Sender". Work may then be submitted to the underlying execution context lazily when `submit` is called on a Sender or eagerly at task creation time, as long as the semantic constraints of the task are satisfied.

2 Background

P0443 presents a unified abstraction for agents of asynchronous execution. It is the result of a long collaboration between experts from many subdomains of concurrent and parallel execution, and achieved consensus within SG1 and, to some degree, LEWG. Although there were known gaps in the abstraction (e.g., reliance on an unspecified `Future` concept), there were papers in flight to address them, and for all intents and purposes P0443 seemed on-target for a TS or, possibly even C++20.

At the Spring 2018 meeting in Rapperswil, a significant design problem was identified: poor support for *lazy* execution, whereby executors participate in the efficient construction and optimal composition of tasks *prior* to those tasks being enqueued for execution. A solution was put forward by P1055 (https://wg21.link/P1055): "senders" (called "deferred" in that paper) and "receivers". Senders and receivers could be freely composed into richer senders and receivers, and only when a receiver was "submitted" to a sender would that task be passed to an execution context for actual execution.

P1055 represented a radical departure from the P0443 design. LEWG deferred the decision to merge P0443 until the alternative had been explored. There are, however, good reasons to be circumspect: a significant redesign now would almost certainly push Executors, and the Networking TS which depends on it, out past C++20. Also, the authors of P1055 had not yet proved to the satisfaction of the experts in SG1 that senders and receivers could efficiently address all the use cases that shaped the design of P0443.

This paper seeks a middle way: a small set of changes to P0443 that improve its support for lazy execution. The changes are limited to such a degree that no functionality has been lost.

In the *ad hoc* Executors meeting in Bellevue, WA on Sept 22-23 2018, some polls were taken regarding overall direction of the Executors design with regards to how best to support the lazy execution scenario. The design presented in this paper (Senders and Reeivers) was endorsed by the experts there.

> **For the long-term direction for executors we like senders/receivers.**
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 9 | 10 | 6 | 2 | 0 |

Note: This paper only seeks to add support for lazy task composition and execution to P0443 while maintaining feature parity with P0443. Companion papers will fill in additional gaps in P0443.

3 Summary of high-level changes

3.1 Parity with P0443

In the interest of maintaining feature parity with P0443 while also adding support for a lazy asynchronous execution model, we suggest the following changes, which are described in more detail below.

1. The placeholder `Future` concept is renamed "`Sender`". A sender may begin executing immediately when the sender is created, or it may defer execution until a continuation is attached.
2. The API for attaching a continuation to a task, which was `then` on the `Future` concept in P0443, is renamed "`submit`" on the `Sender` concept.
3. The `submit` API accepts a `Promise`-like object as a continuation called a `Receiver`. Like `std::promise`, it has separate APIs for setting an error or a value.
4. The `submit` API, unlike `Future.then`, returns `void`.
5. The `then_execute` API on the `Executor` concept is renamed `make_value_task`, and `bulk_then_execute` is renamed `bulk_make_value_task`. (They both return `Sender`s.)
6. The following `Executor` APIs are removed: `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute`.

These changes and the rationale for each are described here.

3.2 Extensions to P0443

1. In addition to `set_error` and `set_value` APIs, the `Receiver` concept gets a `set_done` API by which a sender can notify a receiver that computation is finished and it will not receive either an error or a value. Rationale
2. In addition to `make_value_task` and `make_bulk_value_task` APIs, the `Executor` concept gets a `submit` API that takes a `Receiver` and that passes the executor or a sub-executor to the receiver. That is, an `Executor` is a `Sender` of an executor. Rationale
3. Additionally, the `Sender` concept gets a `get_executor` API for fetching the executor associated with the sender. Rationale

4 Before and After

In the table below, where a future is returned this is represented by the promise end of a future/promise pair in the compromise version. Any necessary synchronization or storage is under the control of that promise/future pair, rather than the executor, which will often allow an entirely synchronization-free structure.

Before	After
`executor.execute(f)`	`executor.execute(f)`
`executor.execute(f)`	`executor.make_value_task(sender{}, f).submit(receiver{})`
`fut = executor.twoway_execute(f)`	`executor.make_value_task(sender{}, f).submit(futPromise)`
`fut' = executor.then_execute(f, fut)`	`executor.make_value_task(fut, f).submit(fut'Promise)`
`executor.bulk_execute(f, n, sf)`	`executor.make_bulk_value_task(sender{}, f, n, sf, []{}).submit(receiver{})`
`fut = executor.bulk_twoway_execute(f, n, sf, rf)`	`executor.make_bulk_value_task(sender{}, f, n, sf, rf).submit(futPromise{})`
`fut' = executor.bulk_then_execute(f, n, sf, rf, fut')`	`executor.make_bulk_value_task(fut, f, n, sf, rf).submit(fut'Promise{})`

Although the “after” here is more verbose, this is a natural side-effect of choosing primitives that are fewer and lowerer-level. It goes without saying that higher-level abstractions can and should be provided to simplify the usage of these primitives.

If a constructed task is type erased, then it may benefit from custom overloads for known trivial receiver types to optimize. If a constructed task is not type erased then the outgoing receiver will trivially inline. That goes likewise for any noop-sender.

We do not expect any performance difference between the two forms of the one way execute definition. The task factory-based design gives the opportunity for the application to provide a well-defined output channel for exceptions. For example, the provided output receiver could be an interface to a lock-free exception list if per-request exceptions are not required.

5 Terminology

This paper describes “task construction” and “task submission”. By the former, we mean creating task dependencies. By the latter, we mean handing the task off to an execution context for execution. We sometimes use the terms “enqueue” or “submit” as synonyms for “task submission.”

A particular executor may decide to perform both task creation *and* task submission in its `make_value_task`. This would be *eager*. Another may decide to only do task creation in its `make_value_task` and do task submission in the `submit` method of the returned `Sender`. That would be *lazy*.

Generic code that uses executors must assume the lazy case and call `submit` on the sender, even though it may do nothing more than attach a (possibly empty) continuation. It must also assume that the work associated with a task may start as eagerly as the call to `make_value_task` itself.

6 Rationale for design changes

6.1 Rationale of Changes to Executor and Future Concepts

The fundamental change suggested by this paper, the one that precipitates all the other changes, is to define the `Future` concept — which was left undefined in P0443 — such that it is a handle to either an eager *or lazy* asynchronous result. Every other change suggested in this paper falls out neatly from that, and most of the changes are simply renames to avoid confusion.

The following describes the changes to the Executors proposal necessitated by the new, weaker semantics of the `Future` concept.

Case 1: Eagerly queued work
: If `Future` is eager, it’s basically `std::experimental::future` (or a non-type-erased equivalent) and everything is as in P0443.

Case 2: Lazily queued work
: If `Future` is lazy, then tasks are enqueued for execution only when `fut.then` is called with a continuation. Why do this? Because a lazy `Future` can be zero overhead. Since there is never a logical race when accessing the continuation, there is no need for synchronization and no need for a dynamically allocated shared state between the promise and future.

1. **Use promises as continuations:**

   P1053 (https://wg21.link/P1053) makes a strong case that the object passed to a `Future`’s `then` API should look like a promise, not like a callable. When chaining operations, a preceding operation may end with an error. Likewise, scheduling work on an execution context may also fail. (Consider the case where `then` successfully schedules work on a threadpool that dequeues work that doesn’t execute within a timeout window.) Those errors need somewhere to go. A promise has an error channel, so it gives us a logical way to chain tasks that propagate both values *and* errors.

2. **Make `then` return `void`:**

   To stay "eager/lazy agnostic", `then` shouldn't be required to return an eager future; return `void` instead, and leave task chaining to `then_execute` on the `Executor` concept.

3. **Rename `then` to `submit`:**

   The name "`then`" doesn't suggest the possibility that it may in fact submit the task for execution. Rename it to "`submit`".

4. **Drop `execute` and `bulk_execute` from the `Executor` concept.**

   Since `then_execute` is no longer required to return an expensive handle to a continuable, eager task, oneway execution can be efficiently built on top of `then_execute`/`submit` by passing a null sender to `then_execute` and a sink receiver to `submit`. We can drop oneway `execute` as a required part of the `Executor` concept and instead provide standard null sender and sink receiver types against which implementations are free to optimize. (The same logic lets us drop `bulk_execute` from the concept.)

5. **Drop `twoway_execute` and `bulk_twoway_execute`:**

   Similarly, `twoway_execute` can be efficiently built on top of `then_execute`/`submit` by passing a null sender to `then_execute` and the promise of an eager future to `submit`, so we can drop `twoway_execute` as a required part of the `Executor` concept (and by extension, `bulk_twoway_execute`).

6. **Rename `then_execute` on the `Executor` concept to `make_value_task`.**

   Since "`then_execute`" really just builds a link in a task chain without necessarily enqueueing it for execution, rename it to "`make_value_task`".

Done. We have now arrived at the executors design suggested by this paper. As we hope the above shows, `Sender` and `Receiver` are really just a minor reformulation of concepts that already appear in P0443 and related papers.

6.1.1 Senders are Futures

Four of the the six `execute` functions from P0443 return a type that satisfies the as-yet-unspecified `Future` concept. A `Future` in the latest draft of P0443, which references the Concurrency TS, is a handle to an already-queued work item to which additional work can be chained by passing it back to an executor's `(bulk_)?then_execute` function, along wih a continuation that will execute when the queued work completes.

A sender is a generalization of a future. It *may* be a handle to already queued work, or it may represent work that will be queued when a continuation has been attached, which is accomplished by passing a "continuation" to the sender's `submit` member function. This is akin to a future that launches work in its `then` API.

6.1.2 Receivers are Continuations

In P0443, a continuation was a simple callable, but that didn't give a convenient place for errors to go if the preceding computation failed. This shortcoming had already been recognized, and was a subject of P1053, which recommended *promises* — which have both a value and an error channel — as a useful abstraction for continuations. A `Receiver` is little more than a promise, with the addition of a channel for a "done" signal.

In short, if you squint at P0443 and P1053, the sender and receiver concepts are already there. They just weren't fully spelled out.

6.1.3 Task Construction is Separate From Submission

The effect of the changes suggested by this proposal is to break the `execute` functions up into two steps:

- Task creation (`s = ex.make_(bulk_)?value_task(...)`), and
- Work submission (`s.submit(...)`).

It is not hard to see how the reformulation is isometric with the original:

```
// Create a task and immediately submit it for execution:
auto fut2 = ex.then_execute(fut1, fn);
```

maps cleanly to:

```
// Create a task:
auto sender2 = ex.make_value_task(sender1, fn);
// Submit it for execution:
sender2.submit(p2);
```

… where `p2` is possibly a promise corresponding to `fut2` from above, though it need not be.

Four of the P0443 `execute` functions return a future. The type of the future is under the executor's control. By splitting `execute` into lazy task construction and a (`void`-returning) work submission API, we enable lazy futures because the code returning the future can rely on the fact that submit will be called later by the caller. With that knowledge, the lazy future is safe to return because we can rely on it being submitted separately.

We optionally lose the ability to block on completion of the task at task construction time. As `submit` is to be called anyway (except for the pure oneway executor case where submit is implicit) it is cleaner to apply the blocking semantic to the invocation of `submit`. In particular, this approach allows us to build executors that return senders that block on completion but are still lazy.

Weakening the Future Concept

Permitting a `Future` to have either eager or lazy submission semantics is a weakening of the concept. Before permitting lazy semantics, the semantics of the `Future` concept were very strong: it was possible to say exactly when work would be enqueued. That strength came at a cost: creating dependent execution chains was expensive, and complexity was needed in the executors to avoid the expense in some limited situations.

Weakening the `Future` concept permits more types to satisfy the concept without the overhead needed to meet the strong semantics. This in turn lets us drop the compexity in executors, which no longer needs to worry about how to magically and efficiently stitch together dependent chains of execution.

6.2 Rationale of Done Signal

In addition to a `set_error` and `set_value` API, the `Receiver` concept gets a `set_done` API by which a sender can notify a receiver that computation is finished and it will not receive either an error or a value.

In acknowlegement of the fact that some tasks will want to support cancellation (by some unspecified means), this paper suggests a mechanism by with a `Sender` can notify a `Receiver` that the task is completing without either an error or a value. This is with a `done` API as part of the `Receiver` concept. *This is not a cancellation mechanism*; rather, it is a means by which a cancellation mechanism can notify downstream computations that they will not execute.

The name `done` is chosen for two reasons: 1. Calling it `cancel` will obviously lead to confusion since this is not a way to cancel an asynchronous task. 2. The `Sender` and `Receiver` concepts naturally extend to multi-value asynchronous computations, also knows as *streams* (not to be confused with C++98 iostreams). In the stream case, a `Sender` calls the receiver's `value` API multiple times, terminating the stream by calling `done` on the `Receiver` to let it know that no additional values are forthcoming. [*Editor's note:* Streams are a planned extension, but are not a part of this proposal. — *end note*]

6.3 Rationale of Executors as Senders of Sub-executors

In a great many interesting scenarios, a launched task needs to know something about the execution agent on which it is executing. Perhaps the task needs to submit nested work to be run on a similar agent, for instance. The exact characteristics of the agent an executor decides to schedule the work on (beyond those explicitly guaranteed by the executor) can be entirely runtime dependent. For instance, a thread pool executor doesn't know *a priori* on which thread it will schedule a task, and that information could be critical for efficient scheduling of nested tasks or correct use of thread-specific state.

In order to keep this information in-band, an `Executor` is a `Sender` whose `submit(...)` API passes itself or some sub-executor through the value channel. In practice, a `Sender` returned from `make_value_task(...)` could work like this:

1. `ex.make_value_task(s1, fn)` returns `s2` that knows about `ex`, `s1`, and `fn`.
2. `s2.submit(r1)` creates a new receiver `r2` that knows about `ex`, `fn`, and `r1` and passes that to `s1.submit(r2)`.
3. If `s1` completes with a value, it calls `r2.set_value(v1)`.
4. `r2.set_value(v1)` builds a new receiver `r3` that captures `fn`, `v1`, and `r1` and passes that to `ex.submit(r3)`.
5. `ex.submit(r3)` makes a decision about where and how to execute `r3` and calls `r3.set_value(subex)`, where `subex` is an executor that encapsulates that decision. (`subex` is possibly a copy of `ex` itself.)
6. `r3.set_value(subex)` now has (a) the value `v1` produced by `s1`, (b) the function `fn` passed to `make_value_task`, and (c) a handle to the execution context on which it is currently running. In the simple case, it simply calls `r1.set_value(fn(v1))`, but it may do anything it pleases including submitting more work to the execution context to which `subex` is a handle.

With this structure, eager executors have the flexibility to create `r2` eagerly but defer the creation of `r3` until the decision about where and how to run the task is made. This is a natural fit for how, for instance, many modern work-stealing schedulers interact with eager dependency expression.

6.4 Rationale for Requiring All Senders to Have Associated Executors

In the executor worldview (even in P0443), all work is carried out on one or more execution agents. While the association of execution agents with work need not happen when that work is created, the association with an entity responsible for the creation of those agents always happens as the work is created. This has not changed from P0443. In this proposal, we represent this association by saying that all `Sender`s have executors. Since Senders are a representation of (potentially deferred) work, their association with an entity responsible for creation of agents to execute that work happens when the Sender is constructed; this is semantically evident from the fact that `make_value_task`, which returns a Sender, is a part of the interface of the executor type.

7 Suggested Design

[Editorial note: The discussed compromise is that we should replace the enqueue functions (`ex.*execute`) with a limited set of (potentially lazy) task factories. Any syntactic cost of providing a trivial output receiver (i.e., a sink) to these operations can be hidden in wrapper functions. We do not expect a runtime cost assuming we also provide a trivial parameter-dropping receiver in the standard library against which an implementation can optimize.

By passing a full set of parameters to task construction functions, any task we place in a task graph may be type erased with no loss of efficiency. There may still be some loss of efficiency if the executor is type erased before task construction because the compiler may no longer be able to see from the actual executor into the passed functions. The earlier we perform this operation, however, the more chance there is of making this work effectively. — *end note]*

7.1 Receiver Concepts

Summary:

- `Receiver<To>`: A type that declares itself to be a receiver by responding to the `receiver_t` property query.
- `ReceiverOf<To, E, T...>`: A receiver that accepts an error of type `E` and the (possibly empty) set of values `T...`. (This concept is useful for constraining a `Sender`'s `submit` member function.)

7.1.1 Customization points

`Receiver` is defined in terms of the following global customization point objects (shown as free functions for the purpose of exposition). These customization points are defined in terms of an exposition-only *`_ReceiverLike`* concept that checks only that a type responds to the `receiver_t` property query. These customization point objects are all declared in the `std::execution` namespace.

Signature	Semantics
`template < _ReceiverLike To >` `void set_done(To& to);`	**Cancellation channel:** Dispatches to `to.set_done` if that expression is well-formed; otherwise, dispatches to (unqualified) `set_done(to)` in a context that doesn't contain the `execution::set_done` customization point object.
`template < _ReceiverLike To, class E >` `void set_error(To& to, E&& e);`	**Error channel:** Dispatches to `to.set_error((E&&) e)` if that expression is well-formed; otherwise, dispatches to (unqualified) `set_error(to, (E&&) e)` in a context that doesn't contain the `execution::set_error` customization point object.
`template < _ReceiverLike To, class... Vs >` `void set_value(To& to, Vs&&... vs);`	**Value channel:** Dispatches to `to.set_value((Vs&&) vs...)` if that expression is well-formed; otherwise, dispatches to (unqualified) `set_value(to, (Vs&&) vs...)` in a context that doesn't contain the `execution::set_value` customization point object.

7.1.2 Concept `Receiver`

```
struct receiver_t {
  static inline constexpr bool const is_requirable = false;
  static inline constexpr bool const is_preferable = false;
};

template <class To>
concept Receiver =
  requires (To& to) {
    execution::query(to, receiver_t{});
    execution::set_done(to);
  };
```

7.1.3 Concept `ReceiverOf`

```
template <class To, class E = exception_ptr, class... Args>
concept ReceiverOf =
  Receiver<To> &&
  requires (To& to, E&& e, Args&&... args) {
    execution::set_error(to, (E&&) e);
    execution::set_value(to, (Args&&) args...);
  };
```

7.2 Sender Concepts

Summary:

- `Sender<From>`: A type that declares itself to be a sender by responding to the `sender_t` property query and that has a `get_executor` API for accessing the sender's executor.
- `TypedSender<From>`: A `Sender` that returns a `sender_desc<E, T...>` descriptor from the `sender_t` property query that declares what types are to be passed through the receiver's error and value channels.
- `SenderTo<From, To>`: A `Sender` and `Receiver` that are compatible.
- `TransformedSender<From, Fun>`: A `TypedSender` and an `Invocable` that are compatible.

7.2.1 Customization points

These customization-points are defined in terms of an exposition-only *_SenderLike<From>* concept that checks only that a type responds to the `sender_t` property query; and *_Executor<Ex>* represents a refinement of `TypedSender` that is a light-weight handle to an execution context, and that sends a single value through the value channel representing another executor (or itself). *_SenderOf<From, T...>* is a `TypedSender` that sends `T...` to its receiver through the value channel. These customization point objects are all declared in the `std::execution` namespace.

Signature	Semantics

`template < _SenderLike From >`
`_Executor&& get_executor(From& from);`
	Executor access:
	asks a sender for its associated executor. Dispatches to `from.get_executor()` if that expression is well-formed and returns a *_SenderLike*; otherwise, dispatches to (unqualified) `get_executor(from)` in a context that doesn't include the `execution::get_executor` customization point object and that does include the following function:
	
	`template< _Executor Exec >`
	`Exec get_executor(Exec exec) {`
	`  return (Exec&&) exec;`
	`}`
	
	Semantics:
	Receivers passed to this sender's `submit` function (see below) will be executed by an agent "owned" by the executor returned by the `get_executor` accessor.

`template < _Executor Exec, Sender Fut, class Fun >`
`requires !TypedSender<Fut> || TransformedSender<Fut, Fun>`
`Sender make_value_task(`
`  Exec& exec,`
`  Fut fut,`
`  Fun fun);`
	Task construction (w/optional eager submission):
	Dispatches to `exec.make_value_task((Fut&&)fut, (Fun&&)fun)` if that expression is well-formed and returns a `Sender`; otherwise, dispatches to (unqualified) `make_value_task(exec, (Fut&&)fut, (Fun&&)fun)` in a context that doesn't include the `execution::make_value_task` customization point object.
	
	Let `T...` be the types that `From` will pass to its receiver through the value channel. The returned sender `S` satisfies `_SenderOf<S, invoke_result_t<Fun, T...>>`, or `_SenderOf<S>` if `invoke_result_t<Fun, T...>` is `void`.
	
	Logically, `make_value_task` constructs a new sender `S` that, when submitted with a particular receiver `R`, effects a transition to the execution context represented by `Exec`. In particular:
	• `submit(S,R)` constructs a new receiver `R'` that wraps `R` and `Exec`, and calls `submit(fut, R')`.
	• If `fut` completes with a done signal by calling `set_done(R')`, then `R'`'s `set_done` method effects a transition to `exec`'s execution context and propagates the signal by calling `set_done(R)`.
	• Otherwise, if `fut` completes with an error signal by calling `set_error(R', E)`, then `R'`'s `set_error` method *attempts* a transition to `exec`'s execution context and propagates the signal by calling `set_error(R, E)`. (The attempt to transition execution contexts in the error channel may or may not succeed. A particular executor may make stronger guarantees about the execution context used for the error signal.)
	• Otherwise, if `fut` completes with a value signal by calling `set_value(R', Vs...)`, then `R'`'s `set_value` method effects a transition to `exec`'s execution context and propagates the signal by calling `set_value(R, fun(Vs...))` — or, if the type of `fun(Vs...)` is `void`, by calling `fun(Vs...)` followed by `set_value(R)`.
	
	Eager submission:
	`make_value_task` may return a lazy sender, or it may eagerly queue work for submission. In the latter case, the task is executed by passing to `submit` an eager receiver such as a `promise` of a continuable `future` so that the returned sender may still have work chained to it.
	
	Guarantees:
	The actual queuing of work happens-after entry to `make_value_task` and happens-before `submit`, when called on the resulting sender (see below), returns.

Signature	Semantics

`template < _Executor Exec, Sender Fut, class Fun,`
`  Invocable ShapeFactory, Invocable SharedFactory,`
`  Invocable ResultFactory >`
`requires !TypedSender<Fut> || TransformedSender<Fut, Fun>`
`Sender make_bulk_value_task(`
`  Exec& exec,`
`  Fut fut,`
`  Fun fun,`
`  ShapeFactory shf,`
`  SharedFactory sf,`
`  ResultFactory rf);`
	Task construction (w/optional eager submission):
	Dispatches to `exec.make_bulk_value_task((Fut&&)fut, (Fun&&)fun, shf, sf, rf)` if that expression is well-formed and returns a `Sender`; otherwise, dispatches to (unqualified) `make_bulk_value_task(exec, (Fut&&)fut, (Fun&&)fun, shf, sf, rf)` in a context that doesn't include the `execution::make_bulk_value_task` customization point object.
	
	Let `T...` be the types that `From` will pass to its receiver through the value channel. The returned sender `S` satisfies `_SenderOf<S, invoke_result_t<Fun, T...>>`, or `_SenderOf<S>` if `invoke_result_t<Fun, T...>` is `void`.
	
	Logically, `make_bulk_value_task` constructs a new sender `S` that, when submitted with a particular receiver `R`, effects a transition to the execution context represented by `Exec`. In particular:
	• `submit(S,R)` constructs a new receiver `R'` that wraps `R` and `Exec`, and calls `submit(fut, R')`.
	• If `fut` completes with a done signal by calling `set_done(R')`, then `R'`'s `set_done` method effects a transition to `exec`'s execution context and propagates the signal by calling `set_done(R)`.
	• Otherwise, if `fut` completes with an error signal by calling `set_error(R', E)`, then `R'`'s `set_error` method *attempts* a transition to `exec`'s execution context and propagates the signal by calling `set_error(R, E)`. (The attempt to transition execution contexts in the error channel may or may not succeed. A particular executor may make stronger guarantees about the execution context used for the error signal.)
	• Otherwise, if `fut` completes with a value signal by calling `set_value(R', Vs...)`, then `R'`'s `set_value` method effects a transition to `exec`'s execution context and propagates the signal as if by executing the algorithm:
	
	`auto shr = sf();`
	`auto res = rf();`
	`for(idx : shf()) {`
	`  fun(idx, t..., sf, rf);`
	`}`
	`set_value(R, std::move(res));`
	
	— or, if the type of `RF()` is `void` or no `RF` is provided, as if by executing:
	
	`auto shr = sf();`
	`for(idx : shf()) {`
	`  fun(idx, t..., sf);`
	`}`
	`set_value(R);`.
	
	Eager submission:
	`make_bulk_value_task` may return a lazy sender, or it may eagerly queue work for submission. In the latter case, the task is executed by passing to `submit` an eager receiver such as a `promise` of a continuable `future` so that the returned sender may still have work chained to it.
	
	Guarantees:
	The actual queuing of work happens-after entry to `make_bulk_value_task` and happens-before `submit`, when called on the resulting sender (see below), returns.

`template < _SenderLike From, Receiver To >` `void submit(From& from, To to);`	**Work submission.** Dispatches to `from.submit((To&&) to)` if that expression is well-formed; otherwise, dispatches to (unqualified) `submit(from, (To&&)to)` in a context that doesn't include the `execution::submit` customization point object. **Guarantees:** The actual queuing of any asynchronous work that this sender represents happens-before `submit` returns.

7.2.2 Concept `Sender`

```
template <class Error = exception_ptr, class... Args>
struct sender_desc {
  using error_type = Error;
  template <template <class...> class List>
  using value_types = List<Args...>;
};

struct sender_t {
  static inline constexpr bool const is_requirable = false;
  static inline constexpr bool const is_preferable = false;
};

// Exposition only:
template <class From>
concept _Sender =
  requires (From& from) {
      execution::query(from, sender_t{});
  };

template <class From>
concept Sender =
  _Sender<From> &&
  requires (From& from) {
      get_executor(from) -> _Sender;
  };
```

7.2.3 Concept `TypedSender`

```
template <class From>
concept TypedSender =
  Sender<From> &&
  requires (From& from) {
    { execution::query(from, sender_t{}) } -> sender_desc<auto, auto...>;
  };
```

7.2.4 Concept `SenderTo`

```
template <class From, class To>
concept SenderTo =
  Sender<From> &&
  Receiver<To> &&
  requires (From& from, To&& to) {
    execution::submit(from, (To&&) to);
  };
```

7.2.5 Concept `TransformedSender`

```
// Exposition only:
template <TypedSender From>
using __sender_desc_of =
  decltype(execution::query(declval<From&>(), sender_t{}));

// Exposition only:
template <class Fun>
struct __transformed_sender_traits {
  template <class... Vs>
  using __invoke_with_t = invoke_result_t<Fun, Vs...>;

  template <TypedSender From>
  using __result_t = typename __sender_desc_of<From>::
    template value_types<__invoke_with_t>;
};

template <class From, class Fun>
concept TransformedSender =
    TypedSender<From> &&
    requires {
      typename __transformed_sender_traits<Fun>::
        template __result_t<From>;
    };
```

7.3 Executor Concepts

An executor should either be a sender of a sub executor, or a one way fire and forget entity. These can be implemented in terms of each other, and so can be adapted if necessary, potentially with some loss of information.

These executor concepts support P0443 style properties (e.g. `require` or `prefer`).

7.3.1 SenderExecutor

A `SenderExecutor` is a `Sender` and meets the requirements of Sender. The interface of the passed receiver should be noexcept.

Function	Semantics
`void submit(ReceiverOf<` *`E`*`, ` *`SubExecutorType`*`> r) noexcept;`	At some future point invokes either: • `set_value(r, get_executor())` on success, or • `set_error(r, err)` on failure, where `err` is object of unspecified type representing an error, or • `set_done(r)` otherwise. *Requires:* `noexcept(set_value(r, get_executor())) && noexcept(set_error(r, err) && noexcept(set_done(r)` is true.
`SenderExecutor get_executor() noexcept`	Returns the sub-executor. [*Note*: Implementations may return `*this`]

`submit` and `get_executor` are required on an executor that has task constructors. `submit` is a fundamental sender operation that may be called by a task at any point.

Invocation of the `Receiver` customization points caused by `submit` will execute in an execution agent created by the executor (the creation of an execution agent does not imply the creation of a new thread of execution). The executor should send a sub-executor to `set_value` to provide information about that context. The sub-executor may be itself. No sub-executor will be passed to `set_error`, and a call to `set_error` represents a failed enqueue. A call to `set_done` indicates that the submission was not accepted (e.g. cancellation).

To avoid deep recursion, a task may post itself directly onto the underlying executor, giving the executor a chance to pass a new sub executor. For example, if a prior task completes on one thread of a thread pool, the next task may re-enqueue rather than running inline, and the thread pool may decide to post that task to a new thread. Hence, at any point in the chain the sub-executor passed out of the executor may be utilized.

7.3.2 OneWayExecutor

The passed function may or may not be `noexcept`. The behavior on an exception escaping the passed task is executor-defined.

Signature	Semantics
`void execute(Function fn)`	At some future point evaluates `fn()` *Requires:* `is_invocable_r<void, Function>` is `true`.

7.4 Ordering guarantees

There are two strong guarantees made here, to allow eager execution:

- Memory operations made before the call to a task constructor happen-before the execution of the task.
- Completion of the task happens-before a call to `set_value` on the next receiver in the chain, including the implicit receiver implied by a task constructor.

A task may therefore run at any point between constructing it, and being able to detect that it completed, for example, by seeing side effects in a subsequent task. This allows tasks to run on construction, in an eager fashion, and does not allow a user to rely on laziness.

The definition of the API does, however, allow laziness and as such no sender is guaranteed to run its contained work until a receiver is passed to its `submit` method (or it is passed to a task constructor that calls `submit` implicitly).

As an optional extension, we may define properties that define whether tasks are guaranteed, or allowed, to run eagerly on construction, assuming that their inputs are ready.

8 Extensions

A wide range of further customization points should be expected. The above description is the fundamental set that matches the capabilities in P0443. To build a full futures library on top of this we will need, in addition:

- A blocking operation for use from concurrent execution contexts. For consistency we suggest defining this as an overload of the `sync_wait` proposal from Lewis Baker, D1171. That paper proposes `std::this_thread::sync_wait(Awaitable)`. We should define a customization point that allows `sync_wait(Sender)` to be implemented optimally.
- A share operation that takes a `Sender` and splits it such that when the input to the `Sender` is complete, multiple `Receivers` may be triggered.
- A join operation that takes multiple `Senders` and constructs a single `Sender` out of them that joins the values in some fashion.
- An unwrap operation that takes a `Sender<Sender<T>>` and returns a `Sender<T>`.

- A task type that supports both value and error handling: `make_value_error_task(Value'(Value), Error'(Error))`, for example.
- A task type that handles only errors and bypasses values: `make_error_task(Error'(Error))` for example.
- Potentially a wider range of parallel algorithm customizations.

By defining these as above in terms of a customization point that calls a method on the executor if that method call is well-formed we can relatively easily extend the API piecemeal with these operations as necessary.

The Sender/Receiver model can be extended to types that asynchronously send and receive more than one value; aka *reactive streams*. That is left as future work.

9 Q&A

9.1 Q: What do lazy execution and eager execution mean?

- *Lazy execution* is when a node in a graph of operations is constructed by storing state and does not start any work.
- *Eager execution* is when a node in a graph of operations starts working as soon as it is constructed.

These can be demonstrated without adding concurrency using normal functions.

Lazy execution:

```
auto f0 = []{ return 40; };
auto f1 = [f0]{ return 2 + f0(); }
// f0 has not been called.

// execute the graph
auto v1 = f1();
// f0 and f1 are complete
```

Eager execution:

```
auto f0 = []{ return 40; };
auto f1 = [auto v0 = f0()]{ return 2 + v0; }
// f0 has been called.

// execute the graph
auto v1 = f1();
// f0 and f1 are complete
```

Without concurrency the only difference is the order of execution. This changes when concurrency is introduced.

9.2 Q: How is eager execution more expensive than lazy execution when concurrency is introduced?

Shared state and synchronization

When concurrency is introduced, the result of an operation is sent as a notification. With coroutines that notification is hidden from users, but is still there in the coroutine machinery.

Often the notification is a callback function. As before, the effect of adding callbacks for eager and lazy execution can be demonstrated without adding concurrency using normal functions.

Lazy execution:

```
auto f0 = [](auto cb0){ cb0(40); };
auto f1 = [f0](auto cb1){ f0([cb1](auto v){ cb1(2 + v); }); }
// f0 has not been called.

// execute the graph
f1([](auto v1){});
// f0 and f1 are complete
```

Eager execution:

```
auto f0 = [](auto cb0){ cb0(40); };
auto sv0 = make_shared<int>(0);
auto f1 = [auto sv0 = (f0([sv0](int v0){ sv0 = v0; }), sv0)](auto cb1){
  cb1(2 + *sv0);
}
// f0 has been called.

// execute the graph
f1([](auto v1){});
// f0 and f1 are complete
```

The shared state is required in the eager case because `cb1` is not known at the time that `f0` is called. In the lazy case `cb1` is known at the time that `f0` is called and no shared state is needed.

The shared state in the eager case is already more expensive, but when concurrency is added to the eager case there is also a need to synchronize the `f0` setting `sv0` and the `f1` accessing `sv0`. This increases the complexity and expense dramatically.

When composing concurrent operations, sometimes eager execution is required. Any design for execution must support eager execution. Lazy execution is almost always the better choice and since lazy execution is also more efficient, lazy execution is the right default.

9.3 Q: In what specific ways was P0443 failing to address the lazy execution scenario?

The Future concept of P0443 was undefined, but the structure of the `then_execute` customization point, which lacked a separate expression of task submission, impeded lazy construction of task chains. Without a separate expression of task submission, the twoway and then_execute functions were making it awkward to build task chains in an executor-specific way without incurring a synchronization and allocation penalty, and impossible with the specific definition of `Future` as `std::experimental::future` in the latest draft. P0443 offered no reasonable way for an executor to participate in the construction and submission of task chains (without, say, resorting to unreasonable measures lake taking advantage of the unspecified behavior of the `Future` destructor to perform task submission).

9.4 Q: What are Senders and Receivers, and how do they help?

Logically, a “Sender” is a handle to an asynchronous computation that may or may not have been submitted for execution yet.

Since a Sender often represents a suspended asynchronous operation, there must be a mechanism to submit it to an execution context. That operation is called “`submit`” and accepts a “`Receiver`”. A `Receiver` is a channel into which a `Sender` pushes its results once they are ready. To be precise, a `Receiver` represents *three* distinct channels: value, error, and done.

It is the `submit` operation that allows Senders and Receivers to efficiently support lazy composition. By contrast, the futures from P0443 and the Concurrency TS had no explicit “submit” operation, only a blocking “`get`” operation and a non-blocking “`then`” operation for chaining a continuation.

If we imagine that a future’s `then` operation which potentially submitted the task to an execution context after the continuation is attached, then we have a lazy `Sender`. Nothing is preventing the `Future` concept from being defined this way, and that is precisely what this paper suggests.

9.5 Q: What makes `std::promise`/`std::future` expensive?

eager execution

When it is possible to call `promise::set_value(. . .)` before `future::get` then state must be shared by the future and promise. When concurrent calls are allowed to the promise and future, then the access to the shared state must be synchronized.

`std::promise`/`std::future` are expensive because the interface forces an implementation of eager execution.

9.6 Q: How is `void sender::submit(receiver)` different from `future<U> future<T>::then(callable)`?

termination

`then` returns another future. `then` is not terminal, it always returns another future with another `then` to call. To implement lazy execution `then` would have to store the callable and not start the work until `then` on the returned future was called. Since `then` on the returned future is also not terminal, it either must implement expensive eager execution or must not start the work, which leaves no way to start the work without using an expensive eager execution implementation of `then`

`submit` returns void. Returning void makes `submit` terminal. When `submit` is an implementation of lazy execution the work starts when `submit` is called and was given the continuation function to use when delivering the result of the work. When `submit` is an implementation of expensive eager execution the work was started before `submit` is called and synchronization is used to deliver the result of the work to the continuation function.

9.7 Q: How is `void sender::submit(receiver)` different from `void executor::execute(callable)`?

signals

`execute` takes a callable that takes `void` and returns `void`. The callable will be invoked in an execution context specified by the executor. Any failure to invoke the callable or any exception thrown by the callable after `execute` has returned must be handled by the executor. There is no signal to the function on failure or cancellation or rejection. When the callable is a functor the destructor will be run on copy, move, success, failure and rejection with no parameters to distinguish them.

`submit` takes a receiver that has three methods.

- `value` will be invoked in an execution context specified by the executor. `value` does any work needed.
- `error` will be invoked in an execution context specified by the executor. any failure to invoke `value` or any exception thrown by `value` after `execute` has returned is reported to the receiver by invoking `error` and passing the error as a parameter
- `done` will be invoked in an execution context specified by the executor. `done` handles cases where the work was cancelled (cancellation is not an error).

9.8 Q: What is the difference between `std::future<T>` and *Sender*?

A crucial difference is in how Sender and Receiver are composed.

- `std::promise<T>` has a `future<T>` that is returned from `std::promise<T>::get_future()`
- A *Sender* has a submit method that takes a *Receiver* and returns void

When the promise is the producer and owns the consumer there are two implementation options.

1. Create shared state and synchronize access to the shared state. This option is the common case as it allows the producer to safely set a value before the consumer asks for the value.
2. Define `std::promise<T>::set_value()` and `std::promise<T>::set_exception()` to have undefined-behaviour when called before the consumer has asked for the value.

When the *Sender* is the producer and the *Receiver* is the consumer then they are both created independently and are bound together by the call to `submit()` on the *Sender*. This form of composition does not prevent a *Sender* from creating a shared state with access to the state synchronized so that binding the *Receiver* later results in the same complexity and behaviour as `std::promise<T>` does, and it also allows a *Sender* to allocate nothing and wait for the *Receiver* to be bound, then call the consumer directly with no synchronization.

10 Acknowledgements

This document arose out of offline discussion largely between Lee, Bryce and David, as promised during the 2018-08-10 executors call.

A great many other people have contrbuted. The authors are grateful to the members of the sg1-exec@ googlegroup for helping to flesh out and vet these ideas.